MEMORANDO TÉCNICO PSet #2: PIPELINE DE DATOS END-TO-END

Nowcasting de ventas sectoriales (46 industrias CIIU) con el Formulario 104 del SRI para el IMAEC Flash

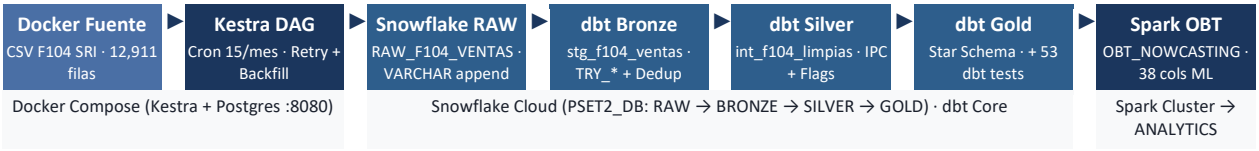
Autor: Ortega Pasmay Edwin | Curso: Ciencia de Datos End-to-End | Stack: Docker · Kestra · Snowflake · dbt Core · PySpark

Resumen ejecutivo.

Este documento presenta la arquitectura, implementación y validación del pipeline ELT reproducible construido para el proyecto del PSet #1: estimar con *nowcasting* las ventas netas mensuales de las 46 ramas de actividad económica (CIIU/CNT) 10 a 15 días después del cierre del mes fiscal, anticipando en 45 a 75 días la publicación del IMAEC oficial del BCE (60 a 90 días de rezago). Todas las cifras provienen de la ejecución real sobre Snowflake (`PSET2_DB`): **12,911 registros** en `RAW`, 138 meses fiscales (`2015-01` a `2026-06`), 46 ramas, dos cortes del Formulario 104 (`temprano_15d` y `definitivo`), **53 pruebas** aprobadas en `dbt build` y **12,635 observaciones** reconciliadas en la One Big Table (`OBT`).

1. Arquitectura e Infraestructura del Pipeline

La arquitectura implementa el patrón ELT moderno con **Snowflake** como único *cloud data warehouse* centralizado. **Kestra** ejecuta la extracción e ingesta programada hacia el esquema `PSET2_DB.RAW`; **dbt Core** gobierna las transformaciones SQL declarativas y las pruebas de calidad a través de las capas `BRONZE`, `SILVER` y `GOLD` (esquema estrella); y **Apache Spark** (`PySpark 3.5.3`) consume las tablas dimensionales desde `GOLD` para construir la tabla analítica ancha (`PSET2_DB.ANALYTICS.OBT_NOWCASTING_VENTAS`). La infraestructura local está declarada como código en `docker-compose.yml` sobre la red bridge `pset2_net`, aislando secretos mediante `.env`.



Flujo end-to-end y desacoplamiento.

(1) El servicio `kestra` (`kestra/kestra:v2.0.7` con backend `postgres:18`) lanza un contenedor efímero `python:3.11-slim` que ejecuta `load_raw_f104.py`, insertando lotes de 5,000 registros en `PSET2_DB.RAW.RAW_F104_VENTAS` con columnas de linaje (`_BATCH_ID`, `_SOURCE_FILE`, `_PERIODO_EJECUCION`, `_LOADED_AT`).

(2) El contenedor `dbt` (`dbt-snowflake`) materializa las capas `BRONZE`, `SILVER` y `GOLD` en 13.3 s y ejecuta las validaciones de contrato.

(3) El cluster `spark-master` (`:8081` / `:7077`) y `spark-worker` (`2 cores`, `2 GB RAM`) ejecuta `spark/build_obt.py` mediante el conector `spark-snowflake_2.12:3.2.3-spark_3.5`, validando cardinalidad antes de publicar la OBT. Dado que `RAW` es inmutable (*append-only*), todas las capas posteriores son 100% deterministas y reconstruibles desde la fuente.

2. Ingesta y Orquestación en Kestra

El flujo declarativo `kestra/flows/f104_to_snowflake_raw.yml` (namespace `ecuador.bce.coyuntura`) automatiza la extracción del archivo semilla `data/seed/f104_ventas_46_ramas.csv` y su carga hacia Snowflake sin intervención manual. La concurrencia está limitada a `concurrency.limit: 1` con estado `QUEUE` para evitar contención de bloqueos en Snowflake durante ejecuciones masivas de *backfill*.

Componente	Configuración técnica y justificación operativa (`f104_to_snowflake_raw`)
Fuente y granularidad	Agregados mensuales del Formulario 104 (IVA) del SRI por rama CIIU (46 industrias) × mes fiscal (`mes_fiscal_id`) × versión de corte (`temprano_15d` a los 15 días y `definitivo`). Incluye `ventas_totales` y `num_declarantes`.
Estrategia en `RAW`	Tabla `PSET2_DB.RAW.RAW_F104_VENTAS` con las 8 columnas de negocio almacenadas como `VARCHAR` sin transformar, preservando el dato crudo exacto. Diseño *append-only* con reconciliación automática (`COUNT(*)` por `_BATCH_ID`).
Trigger y frecuencia	Trigger `io.kestra.plugin.core.trigger.Schedule` con expresión cron `0 6 15 * *` (día 15 de cada mes, 06:00 AM, zona `America/Guayaquil`), sincronizado con el cierre del corte temprano del mes `M-1`. En modo `incremental` recarga `M-1` (nuevo corte temprano) y `M-2` (actualización del corte definitivo con rectificatorias).
Manejo de errores y retries	Política `retry` con `type: constant`, `maxAttempts: 3` e `interval: PT10S` sobre la tarea `cargar_raw` para absorber micro-cortes de red o latencia de reanudación (`AUTO_RESUME`) del warehouse `COMPUTE_WH`. Si los 3 intentos fallan, el bloque `errors` activa `log_error` dejando trazabilidad del período fallido.
Backfill histórico	Parametrizado con la expresión Pebble `{{ trigger.date ?? execution.startDate }}`. Al lanzar un *Backfill* desde Kestra UI sobre un rango histórico, cada ejecución recibe su fecha simulada en `trigger.date` y filtra los meses correspondientes. Para carga inicial completa se dispone del input `modo: completa` y `reemplazar_raw` (`BOOL`).

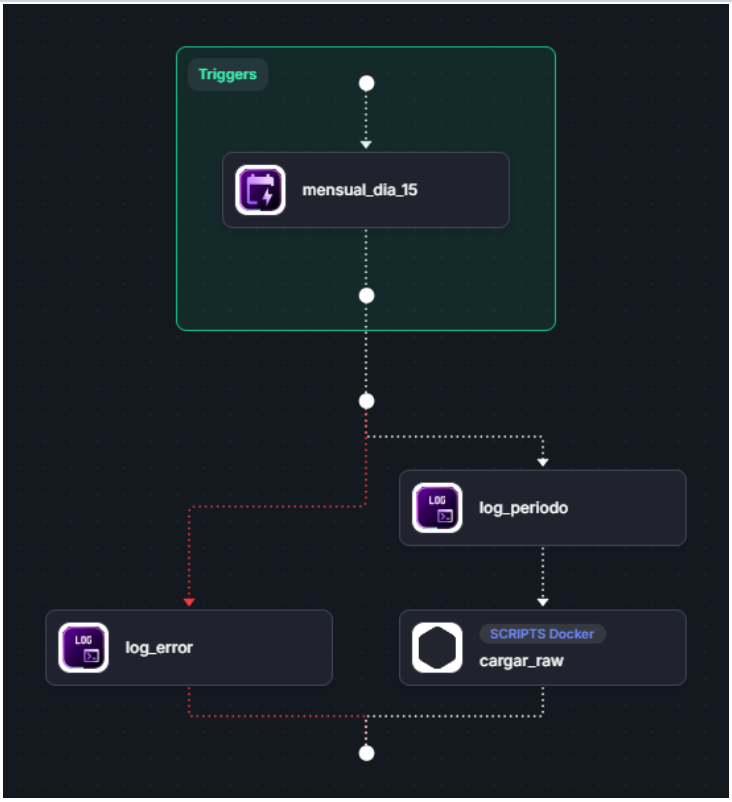


Figura 1. Topología del DAG `ecuador.bce.coyuntura.f104_to_snowflake_raw` validada en Kestra UI.

3. Diagnóstico de Calidad de Datos y Decisiones de Limpieza

Antes de modelar las capas analíticas se perfilaron los **12,911 registros** de `PSET2_DB.RAW.RAW_F104_VENTAS` evaluando cinco dimensiones: **completitud**, **precisión**, **consistencia**, **validez** y **unicidad**. La limpieza estructural se ejecuta en dbt (`stg_f104_ventas` e `int_f104_ventas_limpias`) y se audita en cada compilación mediante **53 pruebas automáticas** (`PASS=53, WARN=0, ERROR=0`).

Dimensión / Problema detectado	Evidencia cuantitativa en RAW	Acción en dbt (Bronze / Silver)	Justificación metodológica y de negocio
Completitud: Código y descripción de rama CIIU nulos (`NULL`)	**276 de 12,911 filas (2.14%)**, exactamente 1 por cada mes (138) y corte (2). Suman USD 1.52B (**0.05%** de las ventas totales).	En `Silver` se asigna el código centinela `'000'` y `es_sin_clasificar = TRUE`; en `Gold` se filtran de `fct_ventas_mensuales`.	Corresponden a RUCs sin CIIU asignado en el catastro del SRI. Imputarlos a una rama sesgaría las 46 series; conservarlos en `Silver` permite conciliar el 100% del total recaudado.
Completitud: Ausencia del corte temprano en pares rama-mes	**61 de 6,348 pares** rama × mes (**0.96%**) carecen de fila `temprano_15d`; el corte `definitivo` está 100% completo (`46 × 138 = 6,348`).	No se rellenan filas sintéticas; `ratio_temprano_definitivo` permanece `NULL` cuando no hubo corte temprano.	Ausencia estructural: ramas pequeñas sin declaraciones en los primeros 15 días. Imputar cero o el promedio histórico induciría sesgo en el entrenamiento del *nowcast*.
Validez: Ventas iguales a cero con declarantes activos (`> 0`)	**57 de 12,911 filas (0.44%)**: 55 en `temprano_15d` y 2 en `definitivo`, concentradas en una sola rama de baja actividad.	Se preservan los ceros reales, se etiquetan con `flag_ventas_cero_con_declarantes` y se protege la división con `NULLIF(..., 0)`.	En el F104 es válido declarar en cero (sin ventas gravadas en el período). Eliminar la fila alteraría la longitud de la serie; `NULLIF` evita divisiones por cero en tasas de variación.
Precisión: Exceso de decimales por aritmética de punto flotante	**4,959 de 12,911 filas (38.41%)** presentan más de 2 decimales (ej. `176422237.689999` en ene-2015, rama `041`).	En `Bronze` se aplica `ROUND(TRY_TO_DOUBLE(ventas_totales), 2)` y casteo explícito a `NUMBER(20,2)`.	La unidad monetaria declarada en el SRI es el centavo de dólar (`0.01`). El redondeo contable elimina el ruido binario IEEE-754 y garantiza comparaciones exactas entre cortes.
Consistencia: Espacios dobles y formato en textos y códigos	**276 filas (2.14%)** en la rama `028` contienen doble espacio interno («`Fabricación de metales...`»); códigos numéricos sin ceros a la izquierda.	En `Bronze`: `REGEXP_REPLACE(TRIM(...), '\\s+', ' ')` para descripciones y `LPAD(..., 3, '0')` sobre `codigo_industria`.	Garantiza que cada una de las 46 ramas colapse en una única entidad dentro de `dim_industria` (`COUNT(*) = 46` validado por test singular).
Consistencia económica: Cobertura temprano vs. definitivo	**0 violaciones** (`temprano > definitivo`) en 6,424 pares. Cobertura media **30.80%** (mediana **29.38%**); **92 filas (1.43%)** con cobertura `< 5%`.	Cálculo de `ratio_temprano_definitivo` en `Silver`, bandera `flag_temprano_mayor_definitivo` y test singular con severidad `warn`.	El corte definitivo acumula las declaraciones tempranas más las tardías; la cobertura media de ~30.8% respalda empíricamente la regla de negocio del PSet #1 (umbral `≥ 30%`).
Unicidad: Duplicados por re-ejecución o *backfill* en `RAW`	**0 duplicados** en el CSV semilla (**0.00%**), pero `RAW` es *append-only* y acumula múltiples *batches* ante reintentos.	En `Bronze`: ventana `ROW_NUMBER() OVER (PARTITION BY fecha_mes, codigo_industria, tipo_corte ORDER BY _loaded_at DESC)` filtrando `rn = 1`.	Asegura idempotencia total: sin importar cuántas veces corra Kestra, `Bronze` retiene únicamente la *vintage* más reciente de cada clave de negocio.
Validez de dominios y rangos: Fechas, conteos y montos no negativos	**0 montos negativos**, **0 errores de parseo** con `TRY_*`, **0 inconsistencias** entre `anio_fiscal/mes_fiscal` y `mes_fiscal_id`. Declarantes entre 1 y `885,911`.	Tests genéricos `not_null`, `non_negative`, `accepted_values` y `value_between` en `models/schema.yml`.	Codifica las restricciones del dominio tributario como contratos ejecutables que detienen el pipeline si ingresa un dato corrupto.

4. Transformaciones en Capas y Modelado Dimensional (dbt Core)

El proyecto dbt (`dbt/dbt_project.yml`) estructura la transformación en tres capas físicas dentro de Snowflake, utilizando el macro `generate_schema_name` para mapear cada capa directamente a su esquema homónimo y gestionando el grafo de dependencias exclusivamente mediante `source()` y `ref()`:

- **Capa Bronze (`PSET2_DB.BRONZE.STG_F104_VENTAS`):** único modelo que invoca `source('raw', 'raw_f104_ventas')`. Ejecuta conversión segura de tipos (`TRY_TO_NUMBER`, `TRY_TO_DOUBLE`, `TRY_TO_DATE`), estandarización de códigos CIIU a tres dígitos (`LPAD`), limpieza de cadenas y deduplicación determinista con `ROW_NUMBER()` por clave natural.
- **Capa Silver (`PSET2_DB.SILVER.INT_F104_VENTAS_LIMPIAS`):** referencia `ref('stg_f104_ventas')`. Aísla los registros sin CIIU bajo el código `'000'`, construye un **deflactor IPC simulado** parametrizado en

`dbt_project.yml` (`ipc_base_fecha = '2018-01-01'`, `ipc_tasa_mensual = 0.0018`, rango `93.73` a `119.92`) para obtener `ventas_reales_usd`, calcula `ventas_por_declarante_usd`, el ratio `ratio_temprano_definitivo` mediante funciones de ventana y cuatro banderas booleanas de coherencia económica (`flag_*`).

- **Capa Gold (`PSET2_DB.GOLD`):** implementa el **Star Schema** separando las medidas aditivas de negocio en una tabla de hechos central y los atributos descriptivos/jerárquicos en tres dimensiones conformadas.

Definición del grano de la tabla de hechos: cada registro de `GOLD.FCT_VENTAS_MENSUALES` representa exactamente **una rama de actividad económica CIIU (`codigo_industria`, 46 ramas) en un mes fiscal (`tiempo_key`, 138 meses) para una versión de corte del Formulario 104 (`corte_key`, 2 cortes)**. Clave subrogada: `venta_key` (hash `MD5` de la clave compuesta). Total: **12,635 filas** (`6,287` en corte `temprano_15d` y `6,348` en corte `definitivo`).



Justificación del Star Schema y pruebas de calidad.

Modelar `dim_corte` como dimensión independiente —en lugar de pivotar ambos cortes en columnas fijas— permite almacenar múltiples *vintages* del mismo mes fiscal y auditar la evolución de la cobertura sin alterar el esquema. A su vez, `dim_industria` agrupa las 46 ramas en 8 sectores macroeconómicos y 3 sectores (`Primario`, `Secundario`, `Terciario`), habilitando agregaciones directas para el informe del IMAEC Flash. La integridad del modelo se verifica con **53 tests** en `dbt build`: `unique` y `not_null` en todas las PKs, `relationships` en las tres FKs de `fct_ventas_mensuales`, `non_negative` en montos y declarantes, y 4 tests singulares (`assert_dim_industria_46_ramas`, `assert_fct_grano_unico`, `assert_fct_concilia_con_silver` y `assert_temprano_no_supera_definitivo`).

5. Construcción de la One Big Table (OBT) con Apache Spark

El job distribuido `spark/build_obt.py` lee `GOLD.FCT_VENTAS_MENSUALES` y las tres dimensiones desde Snowflake, ejecuta los *joins* desnormalizados `N:1` y construye la tabla ancha `PSET2_DB.ANALYTICS.OBT_NOWCASTING_VENTAS` (**12,635 filas × 38 columnas**). **Grano de la OBT:** idéntico al de la tabla de hechos (`codigo_industria × tiempo_key × corte_key`), garantizando una observación por rama, mes y corte lista para el entrenamiento supervisado.

- **Ingeniería de características temporales sin desalineación:** se calcula un índice entero continuo `indice_mes = anio * 12 + mes` y se obtienen los rezagos `lag_1` (t−1) y `lag_12` (t−12) de ventas reales y nominales mediante *self-joins* exactos sobre `(codigo_industria, corte_key, indice_mes + n)`. A diferencia de `LAG()` posicional, este diseño evita que los 61 cortes tempranos faltantes desplacen erróneamente los rezagos.
- **Tasas de variación y representatividad histórica sin fuga temporal (*data leakage*):** se calculan `crecimiento_interanual_real_pct`, `crecimiento_interanual_nominal_pct` y `variacion_mensual_real_pct` usando la función `div_seguro()` (retorna `NULL` cuando el denominador es `0`), además del ratio `ratio_representatividad_corte` y su media móvil

`ratio_representatividad_hist_12m` sobre la ventana `.rangeBetween(-12, -1)` (estrictamente los 12 meses previos, excluyendo el mes actual t). Las columnas `*_definitivo_mes` se incluyen solo como *target*.

Control de integridad en PySpark	Regla de validación ejecutada antes y después de escribir	Resultado en ejecución
Invarianza de cardinalidad post-join	`obt.count() == fct.count()` tras unir `dim_industria`, `dim_tiempo` y `dim_corte` (descarta *fan-out*).	`12,635 == 12,635 [OK]`
Integridad referencial completa	Conteo de filas con `nombre_industria IS NULL OR fecha_mes IS NULL OR tipo_corte IS NULL` igual a cero.	`0 huérfanos [OK]`
Unicidad estricta del grano y PK	`obt.select('codigo_industria','tiempo_key','corte_key').distinct().count() == n_obt` y `venta_key` única.	`12,635 únicas (0 dup) [OK]`
Invarianza tras *self-joins* de rezagos	El número de observaciones permanece idéntico luego de añadir `lag_1`, `lag_12` y la referencia definitiva.	`12,635 filas [OK]`
Reconciliación post-escritura en Snowflake	Lectura de verificación sobre `PSET2_DB.ANALYTICS.OBT_NOWCASTING_VENTAS` comparada contra `n_obt`.	`12,635 == 12,635 [OK]`

Criterio arquitectónico: ¿Cuándo usar el Star Schema (`GOLD`) vs. la OBT (`ANALYTICS`)?

Ambos modelos son complementarios. El **Star Schema** es la capa gobernada para **BI, reportes sectoriales del BCE y consultas ad-hoc**: evita redundancia de atributos descriptivos, permite actualizar catálogos (`sector_macro`) en una sola dimensión (`dim_industria`, 46 filas) y cruzar futuros hechos (VAB trimestral o importaciones) por dimensiones conformadas (`dim_tiempo`, `dim_industria`). La **OBT**, en cambio, es el artefacto óptimo para **Ciencia de Datos y MLOps** (entrenamiento de modelos SARIMA y *Gradient Boosting* con validación *walk-forward* y *scoring* mensual): entrega una vista plana con rezagos y ventanas históricas precalculados de manera idéntica entre entrenamiento e inferencia (evitando *training-serving skew*) sin repetir *joins* ni funciones de ventana en cada experimento.

6. Justificación Arquitectónica: Procesamiento Batch vs. Streaming

Por qué el procesamiento por lotes (*batch*) es la decisión técnica correcta hoy. La elección entre *batch* y *streaming* debe responder a la frecuencia física de generación del dato y al SLA real de la decisión:

- **Dinámica tributaria discreta en la fuente:** el Formulario 104 del SRI se declara mensualmente según el noveno dígito del RUC (días 10 a 28). Por reserva tributaria y convenio interinstitucional, el SRI no expone transacciones en tiempo real, sino **dos cortes agregados por mes** (`temprano_15d` al día 15 y `definitivo` al cierre consolidado).
- **Alineación con la latencia del negocio e intervención humana:** la meta es reducir el rezago del IMAEC oficial de 60–90 días a **10–15 días** tras el cierre mensual. Un pipeline *batch* que corre en **menos de 1 minuto** el día 15 a las 06:00 AM cumple ese horizonte, y la **mesa técnica de analistas** valida outliers antes de publicar el IMAEC Flash.
- **Eficiencia de costos y simplicidad operativa:** mantener un clúster de *streaming* (`Kafka` + `Spark Structured Streaming` + *state stores* 24/7) para procesar ~92 registros al mes añadiría sobrecosto continuo y complejidad de estado (*watermarks*, semántica *exactly-once*) sin mejorar el MAPE del modelo.

¿Qué tendría que cambiar en el caso de uso para justificar una arquitectura de Streaming?

Se requeriría migrar a *streaming* únicamente si convergen tres cambios:

(1) Fuente continua: que el SRI exponga eventos de facturación electrónica (comprobantes autorizados vía API/CDC en línea) en vez de agregados mensuales del F104;

(2) Producto intradía: que el BCE demande un monitor diario/intradía de actividad y alertas en horas ante caídas de cobertura; y

(3) Decisión automatizada: que las predicciones alimenten sistemas automáticos sin revisión humana previa. Incluso entonces, un *micro-batch* diario sobre Snowflake (*Dynamic Tables* / *Streams & Tasks*) sería el paso previo antes de *streaming* puro.

7. Limitaciones Actuales y Riesgos Residuales

Categoría de riesgo	Limitación técnica o de datos aún no resuelta	Plan de mitigación para paso a producción
Gobernanza del SRI y contrato de datos	Por reserva tributaria solo se reciben agregados en CSV sin *Schema Registry*; un cambio unilateral de columnas o códigos CIIU detendría la ingesta.	Validación pre-carga con `dbt source freshness` / `Great Expectations` y cláusula de preaviso técnico de 30 días en el convenio interinstitucional.
Rectificatorias tardías (*Data Leakage*)	Declaraciones sustitutivas modifican meses cerrados; la fuente solo separa `temprano_15d` y `definitivo` sin `fecha_recepcion_sri` por rectificatoria.	Evolucionar `RAW` a un esquema **bitemporal** versionado por `_PERIODO_EJECUCION` para reconstruir *vintages* exactas en el *backtesting walk-forward*.
Drift normativo y sesgo en corte temprano	Prórrogas extraordinarias del SRI (feriados o reformas) alteran quién declara hasta el día 15, desplazando la distribución de `ratio_representatividad_corte`.	Incluir días hábiles efectivos como *feature* y monitoreo mensual de deriva (`PSI` y test `KS`) con *fallback* automático al baseline SARIMA si la cobertura `< 30%`.
Deflactor IPC simulado y RUCs sin CIIU	En `Silver` se aplica un IPC sintético (`+0.18%` mensual desde `2018-01`) y existen **276 filas (2.14%)** sin código CIIU excluidas de `Gold`.	Ingerir en Kestra las series oficiales IPC/IPP del INEC para deflactar por rama y acordar con el SRI la imputación de actividad por RUC histórico.
Arquitectura y operación MLOps	Cluster Spark local de un worker e ingestión desacoplada de la compilación de `dbt` y `Spark`; credenciales en `.env` local y sin CI/CD automatizado.	Encadenar en Kestra el flow de ingesta con subflows para `dbt build` y `spark-submit`, migrar secretos a un *Secret Manager* y activar CI en GitHub Actions.